

Growth Portal Course Proposal Template

Fill in every field before submitting for audit. Fields left blank will be flagged as gaps.

Course Title

Observability Foundations: Structured Logging, Metrics, Tracing, and Failure Detection

1. Background & Strategic Fit

Answer: what documented skill gap does this close, and who besides you needs it? A personal training-plan origin is not sufficient on its own — name the bench/intern/client need this serves.

Closes a documented Growth Area from my own 2-week training plan (observability was the named growth area, retested Day 10). The gap is not unique to me: engineers on other services rely on the same ad-hoc console.log-and-guess workflow this course replaces, and the course is deliberately tiered so QA, UI/UX, and PM roles who work alongside these systems (reading a dashboard, filing a bug against a "slow but not erroring" service) get a jargon-free entry point rather than being excluded.

This does not yet cite a specific incident or measured demand signal beyond the training-plan origin — if Growth Portal wants that before approval, the concrete data point would need to come from Harvey (e.g. a real incident this would have caught faster, or a headcount estimate — see Section 2).

2. Target Learners & Prerequisites

Define the exact starting capability required, not a broad label like "developers." List specific prerequisite skills/tools. State mandatory vs elective and expected headcount.

Target audience

Primary: Full-Stack engineers who ship backend services with no structured observability today — i.e. debugging via ad-hoc console.log and guesswork.

Secondary: QA, UI/UX, and PM roles who work alongside these systems day to day (reading a dashboard, filing a bug report) but don't need to implement any of it themselves. Module 1 is built for this tier specifically — no jargon, no code.

Prerequisites

Everyone (Module 1): none — no code shown, no jargon assumed.

Engineer-deep modules (2-5): comfortable reading/writing a backend service in any language, basic familiarity with running a service locally (e.g. ``docker compose up``), and enough command-line comfort to follow a terminal demo. No prior observability experience assumed — that is precisely what the course teaches.

Mandate

Module 1 (shared/everyone): mandatory — it's a foundational concept regardless of role, not engineering-specific. Modules 2-5 (engineer-deep): mandatory for engineers; elective for QA/UI-UX/PM, who can still watch but aren't required to.

Expected headcount: not yet known — requesting Harvey provide a number for the engineering population this applies to, now that the mandatory scope itself is settled.

3. Duplication Check

Confirm this doesn't overlap with existing Growth Portal content or licensed platforms (Pluralsight, iMocha). If a technology choice is contestable against an industry-standard alternative, justify the choice here.

Checked directly against Growth Portal's current catalog — no existing GP course covers this topic. Not directly checked against Pluralsight or iMocha's catalogs, and that check wouldn't settle much either way: those platforms almost certainly carry a generic observability or structured-logging course, since portable, vendor-agnostic theory is exactly their lane. That's not disqualifying, because this course teaches the opposite: the reasoning behind a proposed, concrete standard — most directly applicable to Node/NestJS-based projects, since that's the stack it's demonstrated in, and offered as a reference rather than an already-adopted org-wide policy — for why a shared-state approach to correlation IDs breaks across an async boundary and what actually fixes it, why an unbounded metric label can quietly blow up in cost, why host-level and app-level metrics answer different diagnostic questions — demonstrated end-to-end inside one real running system rather than described in the abstract. The specific stack (NestJS, Winston, cAdvisor) is the vehicle for showing this concretely, not the subject being taught, and no external platform can show this specific, proposed convention regardless of which vehicle it uses.

Landscape considered and deliberately ruled out of scope, with reasons: alerting/on-call escalation (operational process, not a design concept), SLOs/error budgets (org-level policy, needs a maturity level this codebase doesn't have yet), log aggregation platforms — ELK/Loki/Splunk (tool-specific, not concept-level), APM/RUM (licensed-platform territory, closer to iMocha/Pluralsight's lane), synthetic monitoring and chaos engineering (operational maturity beyond this course's scope), and cardinality as a standalone topic (folded into Module 3 instead of given its own module, since it's only meaningful in the context of a concrete metrics design decision). Kept: structured logging, application metrics + host metrics, distributed tracing, and silent-failure detection — the four topics in Section 6 below.

4. Course Objectives

Use assessable verbs (deploy, configure, diagnose, implement) — not "understand" or "know." State the required proficiency level for each objective. Number them; they will be referenced in the task breakdown, module table, and assessment rubric below.

1. Identify, given a failure scenario, which of the three observability pillars (logs, metrics, traces) would have surfaced it, and explain why the other two would not. [Module 1]
2. Implement structured, correlation-ID-linked logging for a request across an async boundary, and apply the same structured format to a configuration-validation step and an infrastructure connection event. [Module 2]
3. Design a metrics/label scheme for an endpoint that avoids cardinality blow-up, and diagnose from app + host metrics together whether a slow service is compute-bound or I/O-bound. [Module 3]
4. Trace a single request's path across two services using span export, applying the two-gate export rule (end() + flush). [Module 4]
5. Design a dead-man's-switch check for a background job that detects the absence of expected activity, not just the presence of an error. [Module 5]

5. Task Analysis / Breakdown

Break each objective into the component tasks a learner must perform to reach it, before designing modules. Mark each task Terminal (the objective itself) or Enabling (a prerequisite sub-skill), and note dependency order. This is what determines module sequencing in Section 6 — don't skip to modules without it.

Obj #	Task / Sub-skill (the smallest unit a learner must perform)	Type	Depends On (task #)
1.1	Recognize failure symptoms in a system with no observability (before/after narrative)	Enabling	-
1.2	Identify which pillar (logs/metrics/traces) would have surfaced a given failure	Terminal	1.1
2.1	Explain why request-scoped state doesn't survive an `await` without AsyncLocalStorage	Enabling	-
2.2	Identify where a correlation ID should be created and propagated in a request lifecycle	Enabling	2.1
2.3	Implement structured log output (Winston) with a	Terminal	2.1, 2.2

Obj #	Task / Sub-skill (the smallest unit a learner must perform)	Type	Depends On (task #)
	correlation ID across a request handler, config validation, and an infra connection event		
3.1	Identify why an unbounded label (e.g. req.originalUrl) explodes metric cardinality	Enabling	-
3.2	Distinguish counter vs. gauge metric types by use case	Enabling	-
3.3	Design a route-template-labeled metric set for an endpoint, and derive an error ratio without double-counting	Terminal	3.1, 3.2
3.4	Diagnose, from app + host metrics together, whether a slow service is compute-bound or I/O-bound	Terminal	3.3
4.1	Distinguish correlation ID (membership) from a span tree (structure) in a trace	Enabling	2.2
4.2	Trace one request's path across two services using span export (end() + flush)	Terminal	4.1
5.1	Distinguish an absence-of-success signal from a presence-of-error signal	Enabling	-
5.2	Design a dead-man's-switch check for a background job that alarms on missed heartbeats	Terminal	5.1

6. Proposed Modules

Each module needs its own outcome, activity, assessment, prerequisite, and duration — not just a topic name. The Obj # column must trace back to Section 4; a module with no objective number is a module nobody asked for.

#	Module Title	Obj #	Learning Outcome	Activity / Format	Assessment	Prerequisite	Duration
1	Why observability, and the 3 pillars	1	Identify which pillar would surface a given failure	Video: before/after narrative on coworking-obs, no code, no jargon	Whole-course GP post-assessment (Section 10)	None	~5 min
2	Structured logging + correlation IDs	2	Implement correlated structured logging across a request and beyond it	Video: mechanism explained, then demonstrated in coworking-obs (ClsMiddleware, config validation, infra connection log)	Whole-course GP post-assessment (Section 10)	Module 1	~12 min
3	Metrics that survive scale	3	Design cardinality-safe metrics; diagnose compute- vs. I/O-bound slowness	Video: mechanism explained, then demonstrated with cAdvisor + Grafana panel, incl. wiring diagram	Whole-course GP post-assessment (Section 10)	Module 1	~15 min
4	Distributed tracing	4	Trace a request across two services via span export	Video: correlation ID vs. span tree, then demonstrated against the Day 6 tracing setup	Whole-course GP post-assessment (Section 10)	Module 1, Module 2	~10 min

#	Module Title	Obj #	Learning Outcome	Activity / Format	Assessment	Prerequisite	Duration
5	Detecting silent failure	5	Design a dead-man's-switch check for a background job	Video: absence-of-success vs. presence-of-error, then demonstrated against the Day 8 cron	Whole-course GP post-assessment (Section 10)	Module 1	~10 min

7. Instructional Strategy & Delivery Format

State the delivery method: self-paced, facilitated, cohort, or 1:1 with a mentor. State total course duration and estimated time per module.

Self-paced, recorded video, posted to Growth Portal — not facilitated, not cohort-based, not 1:1 mentoring. Each module is tier-labelled ("for everyone" / "assumes engineering background") so a learner can self-select depth. Total estimated course duration: ~50-55 minutes across 5 modules (Section 6 has the per-module breakdown). These are estimates pending the Day 2-4 read-aloud checks against locked scripts; treat them as directional, not final, until then.

8. Materials & Resources

List planned materials beyond the infra/tool stack: slides, video, written docs, reference links.

Recorded video (primary deliverable), tier-labelled per module.

One wiring diagram (Module 3): host-metrics exporter -> Prometheus -> Grafana, since no single file in the demo app shows that relationship in one place.

Demo app repository link (coworking-obs) referenced in the video description, for anyone who wants to read the actual code rather than just watch it demonstrated.

GP-native pre/post-assessment question bank (Section 10) — authored Day 7, after all 5 module scripts lock, so it tests real content rather than a draft outline.

9. Hands-On Environment

Describe the infrastructure. Confirm the environment can actually support every promised objective (e.g. don't promise high availability on a single-node setup). Document networking/security for any external infrastructure, and how the environment will be provisioned for learners beyond you.

No learner-side environment to provision. This is video-only delivery — a learner watches, they don't run anything, so there is nothing to stand up, secure, or network on their end.

The demo app (coworking-obs: NestJS + Postgres + a host-metrics exporter, run via docker-compose) exists only on the presenter's machine, for the purpose of recording the demonstration segments. It is not provisioned for or distributed to learners, and no learner ever needs network or security access to it.

This sidesteps the infra-provisioning problem that affects hands-on-lab-style proposals: there is no environment to keep working for other people after launch, only a video.

10. Assessment Plan / Capstone

Every objective in Section 4 must map to at least one row below. List specific, observable pass criteria — not just a list of activities.

Pass Criterion (specific, observable, checkable)	Maps to Objective #	Pass / Fail Threshold
Learner correctly identifies which pillar would have surfaced a given failure scenario	1	Proposed: 70% overall pass mark on the whole-

Pass Criterion (specific, observable, checkable)	Maps to Objective #	Pass / Fail Threshold
		course post-assessment (not per-objective) — confirm with Harvey
Learner correctly identifies the structured-logging fix for a given async correlation-loss scenario	2	Included in the 70% overall whole-course threshold above
Learner correctly picks the cardinality-safe label design, or the compute- vs. I/O-bound diagnosis, for a given scenario	3	Included in the 70% overall whole-course threshold above
Learner correctly orders a request's span tree across two services from a given trace snippet	4	Included in the 70% overall whole-course threshold above
Learner correctly distinguishes an absence-of-success case from a presence-of-error case in a given scenario	5	Included in the 70% overall whole-course threshold above

11. Mid-Course Feedback Checkpoints

A final capstone alone is too late to catch a learner drifting off track. Define at least one checkpoint before the final assessment.

Open question for Harvey: does Growth Portal support a native mid-course or per-module assessment mechanism, or only a whole-course pre/post pair? If it doesn't, the proposal is to either settle for whole-course-only assessment (Section 10), or build a lightweight non-GP workaround (e.g. an external, ungraded form) for Module 3 specifically. If GP does support it, a native mid-course check placed after Module 3 would be worth adding directly.

Module 3 (host-metrics addition, live diagram narration) is the highest-risk module for learner confusion — whichever way this goes, it's the module a checkpoint should cover if one gets built.

12. Success Metric & Reporting

Define the completion/mastery signal and how it rolls into Rocks (checkpoint ratings, bench action plan tracking). A course with no reporting hook is invisible to EM oversight.

Completion signal: GP-native post-assessment pass/fail, tracked and graded by GP itself (Section 10).

Rocks/checkpoint-tracking hookup: open question for Harvey, narrowed to three specific unknowns rather than one vague one. Rocks is a separate internal Full Scale tool tracking employee performance broadly, but: (1) whether or how Rocks integrates with Growth Portal course completions at all is outside my visibility, (2) there's no known precedent of another GP course already having a Rocks hookup to model this on, and (3) even the definition of "success" for hookup purposes (completion vs. passing the post-assessment threshold) hasn't been settled.

Requesting Harvey's input on whether a hookup is technically possible at all, and if so, what mechanism or precedent to follow.

13. Maintenance Owner

Name who owns keeping this content current after launch. Technical content ages quickly.

Leander (course author). Covers the demo app's dependencies as they drift — Postgres, env-schema validation, the host-metrics exporter (cAdvisor) and its Grafana provisioning — and refreshing the video content itself if the underlying concepts or the demo app's structure change materially.

14. Execution Plan

Include design review, technical validation, pilot delivery, and a revision cycle — not just a content-production schedule.

Date / Window	Activity	Owner	Acceptance Criteria
Pre-Day 0 (this proposal review)	Design review with EM	Harvey	Audit gate items F1-F7 pass (or have an explicit resolution plan); template approved to proceed to build
Day 0	Technical/content validation	Leander	docker-compose stack runs; verify.sh green; health-check endpoint and new Grafana panel confirmed working
Days 1-6	Content build	Leander	All 5 module scripts locked and read-aloud checked; all 5 modules recorded
Day 7 end / Day 8 start	Pilot delivery (1-2 learners)	Leander	1-2 volunteer engineers watch the edited, unpublished video and give feedback in parallel with the EM review below; no blocking confusion reported on Module 3
Day 8	Revision based on pilot feedback	Leander	Pilot feedback and EM feedback both incorporated before publish
Day 8 end	Publish to Growth Portal	Leander	Video live on GP with tier labels; pre/post-assessment active and graded

15. Expected Outcome

State what learners will be capable of doing after the course, in one measurable sentence — not what the course will contain.

After completing this course, a learner can: read a system's logs, metrics, and traces to explain what a request actually did; add a structured, correlated log line at a request handler, a config-validation step, or an infrastructure connection event; design a metric label set that won't blow up in cardinality; distinguish a compute-bound slowdown from an I/O-bound one using app vs. host metrics; follow one request's path across two services via distributed tracing; and design a dead-man's-switch check that catches a job that silently stops running.